Doc. no.:	P3072R1
Date:	2024-2-15
Audience:	LEWG
Reply-to:	Zhihao Yuan <zy@miator.net>

Hassle-free thread attributes

Changes

Since R0

- Add the wording

Introduction

P2019R5[1] proposes to allow specifying thread attributes, such as name and stack size, when constructing `std::thread` and `std::jthread`. In such an approach, users express the attributes as objects, one type per attribute. P3022R1[2], with a mind to standardize the existing practices, groups all standard attributes into one type. In both papers, the types to represent

attributes are implementation-defined. This paper proposes a fully specified aggregate to represent all the thread attributes defined by the standard. The vendors can have extra types to carry more or different attributes, as this paper ensures that the different types require differently looking codes.

Here is a comparison of the major use cases of the three papers:

P2019

```
std::jthread thr(std::thread::name_hint("worker"),
                 std::thread::stack_size_hint(16384),
                 [] { std::puts("standard"); });
```

```
std::jthread thr(__gnu_cxx::posix_schedpolicy(SCHED_FIFO),
                 [] { std::puts("vendor extension"); });
```

P3022

```
std::jthread::attributes attrs;
attrs.set_name("worker");
attrs.set_stack_size_hint(16384);

std::jthread thr(attrs, [] { std::puts("standard"); });
```

```
__gnu_cxx::posix_thread_attributes attrs;
attrs.set_schedpolicy(SCHED_FIFO);

std::jthread thr(attrs, [] { std::puts("vendor extension"); });
```

P3072

```
std::jthread thr({.name = "worker", .stack_size_hint = 16384},
                 [] { std::puts("standard"); });
```

```
std::jthread thr(__gnu_cxx::posix_thread_attributes{
                     .schedpolicy = SCHED_FIFO,
                 },
                 [] { std::puts("vendor extension"); });
```


Motivation

P2019R5[1:1] has provided excellent motivation for standardizing thread attributes. The rest of this section will focus on the additional motivation for specifying these attributes in an aggregate.

Enjoy familiar, natrual, and terse syntax

Attributes are declarative data to most of the programmers. "No two attributes can be of the same type" is an unheard-of restriction, and "calling a subroutine to specify an attribute" is a ubiquitous complication. Designated initializers are already familiar to the C++ users. They precisely specify the attributes, are naturally isolated from the other arguments by the surrounding braces, and are as terse as possible.

Be trivially ABI stable

Changes that can break an aggregate's ABI are apparent and often break API simultaneously; therefore, we won't make them. We never argue whether `std::from_chars_result` is ABI stable. The standard thread attribute aggregate, i.e., `std::thread::attributes` in this paper, is meant to be as stable as `std::from_chars_result`.

Discussion

The proposed content is compact enough to fit in here for further discussion. First, add a new inner class `attributes` in the scope of `std::thread`.

```
class thread
{
  public:
    struct attributes
    {
        std::string const &name = {};
        std::size_t stack_size_hint = 0;
    };

    …
```

And then, add extra constructors to `std::thread` and `std::jthread`, one for each.

```
class thread
{
    …

    template<class Attrs = attributes, class F, class... Args>
        requires std::is_invocable_v<F, Args...>
    explicit thread(Attrs, F &&, Args &&...);

    …
};

class jthread
{
    …
    using attributes = thread::attributes;

    template<class Attrs = attributes, class F, class... Args>
        requires std::is_invocable_v<F, Args...>
    explicit jthread(Attrs, F &&, Args &&...);

    …
};
```

They enable a variety of uses.

Specifying all standard thread attributes

```
std::thread t1({.name = std::format("worker {}", i), .stack_size_hint = 163
                [] { std::puts("everything"); });
```

As designators, neither attribute can repeat in the same list; `.name` must precede `.stack_size_hint` if both appear.

Specifying only the thread name

```
std::thread t2({.name = "gui"}, [] { std::puts("only name"); });
```

`stack_size_hint` has no effect if it compares equal to 0.

Providing only a hint to the stack size

```
std::thread t3({.stack_size_hint = 4096}, std::puts, "only size");
```

`name` has no effect if it is an empty string.

Declaring the attributes object as a variable

```
std::thread::attributes attrs{.name = std::format("worker {}", i)};
std::thread t4(attrs, std::puts, "lifetime extension");
```

Member of reference type extends the lifetime of its initializer in a braced aggregate initialization. Replacing the braces with parenthesizes loses lifetime extension, but designated initializers cannot appear inside parenthesizes in the first place.

Substituting in non-standard thread attributes

```
std::thread t5(__gnu_cxx::posix_thread_attributes{.schedpolicy = SCHED_FIFO
               std::puts, "vendor extension");
```

The users cannot omit the type names for the non-standard attributes in front of the *braced-init-list*.

`std::jthread` offers the same capability.

Technical Decisions

Make `thread::attributes` platform-independent

In a survey from P3022[2:1], Boost.Thread stores OS-provided thread attribute handle in `boost::thread::attributes` on certain platforms. More specifically, `pthread_attr_t` on POSIX. This may give the audiences a misconception – the thread name may be managed by an opaque type so that a standard library implementation doesn't need to allocate anything extra for that string. This is not true. A table in P2019 shows that no platform supports an attribute handle of that design. And `Boost.Thread` only supports setting the stack size, which is the least motivating attribute to be represented platform-dependently.

Since thread names often come with a 15-character limit on non-Windows platforms, do we want different guts when implementing the name attribute on different platforms, then?

```
#if defined(_WIN32)
    unique_ptr<char[]> name_;
#else
    char name_[16];
#endif
```

It turns out that `std::string` offers this, and only better. Therefore, using `std::string` in some form is entirely acceptable when specifying the thread name.

The motivation for making **standard** thread attribute types implementation-defined is weak. A `thread::attributes` that reuses existing standard library types brings less hassle when working with. Consequently, `thread::attributes` should be platform-independent in a given standard library implementation.

Declare the `name` attribute as `string const&`

In a prior discussion of P2019, LEWG concluded that thread attributes may depend on runtime values in many cases, such as thread names formatted with a counter. In today's C++ standard, the best practice for creating strings of that kind is to use `std::format`. Therefore, the full solution to thread attributes must be optimal when combined with `std::format`.

If the `name` member of `std::thread::attributes` is of type `char const*`, one must call `.c_str()` or an equivalent member function on the result of `std::format`. The code to initialize is not only bumpy but also invites dangling pointers:

```
std::thread::attributes attrs{.name = std::format("worker {}", i).c_str()}; // dang
```

If `name` is declared `string_view`, the dangling error hides even better:

```
std::thread::attributes attrs{.name = std::format("worker {}", i)};    // also dang
```

Moreover, as a survey from P2019 suggested, all platforms expect thread names to be null-terminated. `string_view` does not guarantee its content to be null-terminated and requires extra work in the `thread` and `jthread` constructors.

If `name` is declared `string`, the last code snippet won't be dangling. However, the size of the `attributes` struct will be doubled. The type won't be trivial, will require more work to be moved or copied, and can easily trigger a copy:

```
auto launch_first(std::vector<std::string> const& names)
{
    return std::thread({.name = names.front()}, this);
}
```

Declaring `name` as `string const&` has none of the aforementioned issues.

Default a template parameter to `thread::attributes`

Assigning the default argument of a template parameter `T` to an aggregate enables braced initialization of an argument for a function parameter of type `T` without filling out the type name of the aggregate at the caller site. Declaring that function parameter as the aggregate has the same effect. So, can we allow vendor extensions by building an overload set?

P3072	`template<class Attrs = attributes, class F, class... Args>`
	`  requires is_invocable_v<F, Args...>`
	`explicit thread(Attrs, F &&, Args &&...);`

Alt.	`template<class F, class... Args>`
	`explicit thread(attributes, F &&, Args &&...);`
	
	`template<class F, class... Args>`
	`explicit thread(__extended_thread_attributes, F &&, Args &&...);`

Unsurprisingly, there is a critical difference. If `__extended_thread_attributes` is declared like this,

```
struct __extended_thread_attributes
{
    char name[16];
    size_t stack = 0;
    int schedpolicy = SCHED_OTHER;
};
```

the following code will become ill-formed with the alternative design because the call is ambiguous.

```
std::thread t2({.name = "gui"}, [] { std::puts("only name"); });
```

If we were to pursue this design, name conflicts between the standard thread attributes, vendor extended attributes, and future standard thread attributes would have to be resolved at the member level.

The proposed design avoids this type of ambiguity by construction. The declarion above initializes only `std::thread::attributes`.

Alias `jthread::attributes` to `thread::attributes`

Doing so won't be the reason that prevents us from evolving `jthread::attributes` independently from `thread::attributes` because adding a member to a public aggregate defined in the standard is not an option to begin with. This also means if we do want the content of `jthread::attributes` and `thread::attributes` to diverge, the decision must be made now. But as time of writing, we see no motivation in such a direction.

Allowing a variable declared `std::thread::attributes` to be shared by both `std::thread` and `std::jthread` constructors looks entirely acceptable. This also avoids the headache of debating the effect of the following code:

```
std::thread::attributes attrs{.name = std::format("worker {}", i)};
std::jthread t(attrs, std::puts, "ignored, ill-formed, or vendor extension?");
```

Implementation

P2019R5[1:2] has discussed the implementability of platform-independent thread names and stack size hints.

Here is a playground to demonstrate the proposed interface of this paper: Compiler Explorer jban3hG37

Combining these should give us a complete implementation.

Wording

The wording is relative to N4971.

In [thread.thread.class.general], modify the synposis in the first paragraph as follows:

```
namespace std {
  class thread {
  public:
    // [thread.thread.id], class thread::id
    class id;
    using native_handle_type = implementation-defined;  // see [thread.req.native]
    struct attributes {
        string const& name = {};
        size_t stack_size_hint = 0;
    };

    // construct/copy/destroy
    thread() noexcept;
    template<class F, class... Args> explicit thread(F&& f, Args&&... args);
    template<class Attrs = attributes, class F, class... Args>
      explicit thread(Attrs, F&&, Args&&...);
    ~thread();
```

Append the following paragraph after [thread.thread.class.general]/1:

> The type `attributes` have the data members, their default member initializers, and special members specified above. They have no base classes or members other than those specified.

Modify [thread.thread.constr] as follows:

```
thread() noexcept;
```

> *Effects:* The object does not represent a thread of execution.
>
> *Postconditions:* `get_id() == id()`.

```
template<class F, class... Args> explicit thread(F&& f, Args&&... args);
template<class Attrs = attributes, class F, class... Args>
  explicit thread(Attrs attrs, F&& f, Args&&... args);
```

Constraints: `remove_cvref_t<F>` is not the same type as `thread` and `is_invocable_v<decay_t<F>, decay_t<Args>...>` is `true`.

Mandates: The following are all `true`:

- `is_constructible_v<decay_t<F>, F>`,
- `(is_constructible_v<decay_t<Args>, Args> && ...)` ~~, and~~
- ~~`is_invocable_v<decay_t<F>, decay_t<Args>...>`~~.

Effects: The new thread of execution executes

```
invoke(auto(std::forward<F>(f)),  // for invoke, see [func.invoke]
       auto(std::forward<Args>(args))...)
```

with the values produced by `auto` being materialized ([conv.rval]) in the constructing thread. Any return value from this invocation is ignored.

[Note 1: This implies that any exceptions not thrown from the invocation of the copy of `f` will be thrown in the constructing thread, not the new thread. *–end note]*

If the invocation of invoke terminates with an uncaught exception, terminate is invoked ([except.terminate]).

The parameter `attrs` can be used to tailor the new thread with implementation-defined properties that do not alter the semantics of its execution. An implementation may accept `attrs` of additional types other than `thread::attributes`.

[Note 2: When `attrs` is of type `thread::attributes`, an implementation may derive the name for the new thread for debugging or platform-specific display mechanisms from `attrs.name` and may set an initial stack size based on `attrs.stack_size_hint`. *–end note]*

Recommended practice: Implementations should avoid storing the value of `attrs` in the `thread` object.

[...]

In [thread.jthread.class.general], modify the synposis as follows:

```
namespace std {
  class jthread {
  public:
    // types
    using id = thread::id;
    using native_handle_type = thread::native_handle_type;
    using attributes = thread::attributes;

    // [thread.jthread.cons], constructors, move, and assignment
    jthread() noexcept;
    template<class F, class... Args> explicit jthread(F&& f, Args&&... args);
    template<class Attrs = attributes, class F, class... Args>
      explicit jthread(Attrs, F&&, Args&&...);
    ~jthread();
```

Modify [thread.jthread.cons] as follows:

```
jthread() noexcept;
```

> *Effects:* Constructs a `jthread` object that does not represent a thread of execution.
>
> *Postconditions:* `get_id() == id()` is `true` and `ssource.stop_possible()` is `false`.

```
template<class F, class... Args> explicit jthread(F&& f, Args&&... args);
template<class Attrs = attributes, class F, class... Args>
  explicit jthread(Attrs attrs, F&& f, Args&&... args);
```

Constraints: `remove_cvref_t<F>` is not the same type as `jthread` and `is_invocable_v<decay_t<F>, decay_t<Args>...> ||` `is_invocable_v<decay_t<F>, stop_token, decay_t<Args>...>` is `true`.

Mandates: The following are all `true`:

- `is_constructible_v<decay_t<F>, F>`,
- `(is_constructible_v<decay_t<Args>, Args> && ...)` ~~, and~~
- ~~`is_invocable_v<decay_t<F>, decay_t<Args>...> ||`~~ ~~`is_invocable_v<decay_t<F>, stop_token, decay_t<Args>...>`~~.

Effects: Initializes `ssource`. The new thread of execution executes

```
invoke(auto(std::forward<F>(f)), get_stop_token(),  // for invoke, see [func.invoke]
       auto(std::forward<Args>(args))...)
```

if that expression is well-formed, otherwise

```
invoke(auto(std::forward<F>(f)), auto(std::forward<Args>(args))...)
```

with the values produced by `auto` being materialized ([conv.rval]) in the constructing thread. Any return value from this invocation is ignored.

[Note 1: This implies that any exceptions not thrown from the invocation of the copy of `f` will be thrown in the constructing thread, not the new thread. *–end note]*

If the `invoke` expression exits via an exception, `terminate` is called.

The parameter `attrs` can be used to tailor the new thread with implementation-defined properties that do not alter the semantics of its execution. An implementation may accept `attrs` of additional types other than `thread::attributes`.

[Note 2: When `attrs` is of type `thread::attributes`, an implementation may derive the name for the new thread for debugging or platform-specific display mechanisms from `attrs.name` and may set an initial stack size based on `attrs.stack_size_hint`. *–end note]*

Recommended practice: Implementations should avoid storing the value of `attrs` in the `jthread` object.

[…]

References

1. P2019R5 *Thread attributes.*
   https://wg21.link/p2019r5 ↩ ↩ ↩

2. P3022R1 *A Boring Thread Attributes Interface.*
   https://wg21.link/p3022r1 ↩ ↩